

AXI STREAM COMPRESSOR/PACKER

Theoretical background

Packing (also known as null-byte stripping) is the process of removing invalid padding bytes from an AXI4-Stream data word, guided by the **keep** byte qualifier, to produce a dense, gap-free data stream.

Rather than encoding or altering the payload, packing simply discards bytes that carry no information and contiguously realigns the remaining valid data toward the Least Significant Bit (LSB). This optimization ensures that the downstream interface maximizes its bandwidth on useful payload instead of wasting cycles on empty padding.

[AXI stream specification.](#)

Goal

Your task is to design an IP that receives an AXI-Stream packet containing null bytes and outputs the same packet with those null bytes removed.

Provided module

You can edit only the *task_16.sv* file. If you wish, you may add other files for supportive submodules.

The module has the following inputs and outputs:

Table 1 List of inputs and outputs of the task_16 module.

Signal name	Signal type	Bit length
i_clk	Input	1 bit
i_rst	Input	1 bit
i_valid	Output	1 bit
l_ready	Input	1 bit
i_data	Input	32 bits
i_keep	Input	4 bits
i_last	Input	1 bit
o_valid	Output	1 bit
o_ready	Input	1 bit
o_data	Output	32 bits
o_keep	Output	4 bits
o_last	Output	1 bit

The reset is synchronous and active high.

Input and output interfaces

In one packet task receives **from 4 to 512 samples** of input data (i_data), 32 bits each. The 4-bit i_keep signal qualifies i_data , specifying which bytes contain valid data and which represent padding (null bytes) to be discarded.

Each bit $i_keep[x]$ is directly associated with the corresponding byte lane $i_data[(8x+7):8x]$, where a high level ('1') indicates a valid byte and a low level ('0') denotes a null padding byte.

The valid i_keep patterns are:

{0x1, 0x3, 0x7, 0xf}

These patterns are intended to simplify the shifting/packing logic: the gaps (null bytes) are always located at the MSB side.

The last sample in a packet is indicated by signal i_last .

The task should send N_{out} of output samples, 32 bits each, and should indicate the last sample in the output packet by providing signal o_last .

$$N_{out} = \left\lceil \frac{\sum_{k=1}^{N_{in}} \text{popcount}(i_keep_k)}{4} \right\rceil$$

Where:

1. N_{in} is the total number of input beats in the incoming packet
2. i_keep_k is the 4 bit keep mask of the k -th input sample.
3. $\text{popcount}(i_keep_k)$ is the number of bits set to 1 (valid bytes) in that mask (ranging from 0 to 4).
4. $\sum \text{popcount}(\dots)$ represents the total number of valid bytes in the entire packet.
5. $\lceil \dots \rceil$ is the ceiling function, rounding up to the nearest integer, since even a single remaining byte requires a full 32-bit transfer slot at the output.

The o_keep signal must be set to 0xF for every output sample, except for the last sample of a packet when the total number of valid bytes is not a multiple of 4. In that case, o_keep reflects only the valid bytes remaining in the final transfer.

Backpressure handling

Both the input and output interfaces use the AXI-Stream valid/ready handshake and the design must fully support backpressure. On the output side, *o_ready* may be deasserted at any time; while *o_ready* is low, the module must keep *o_data*, *o_keep*, *o_valid* and *o_last* stable and must not drop or duplicate any data. On the input side, the module drives *i_ready* to stall the upstream source whenever it cannot accept a new beat (for example while its internal buffer is full). A transfer happens only on cycles where both valid and ready are asserted.

Example 1:

Input interface:

i_data – 0xa3a2a1a0, 0xb3b2b1b0, 0xc3c2c1c0, 0xd3d2d1d0

i_keep – 0x1, 0x7, 0x3, 0xf

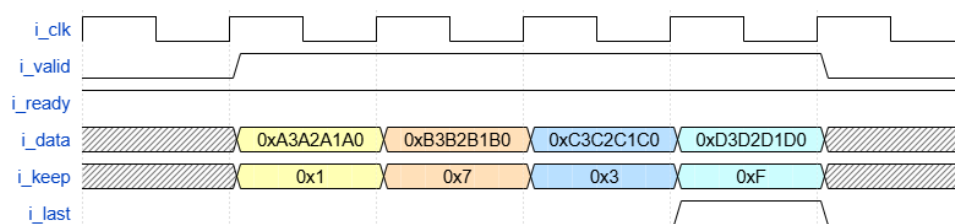


Figure 1 Example input packet.

Output interface:

o_data – 0xb2b1b0a0, 0xd1d0c1c0, 0xFFFFd3d2

X means that it doesn't matter what value is on those bits

o_keep – 0xf, 0xf, 0x3

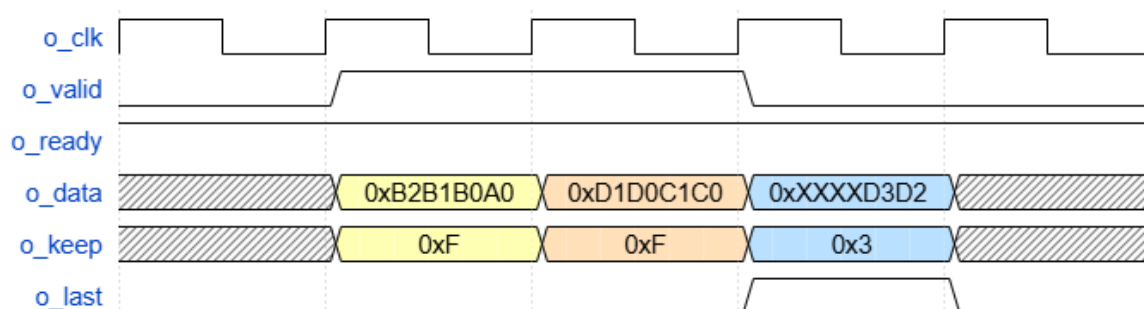


Figure 2 Example output packet.

Evaluating the task

The evaluation is done by comparing the data returned by the task with the expected sequence calculated by the reference model in the Judge. The sequence is checked for both its length and content, and if it is the same, the task is marked as correctly solved.

In Development mode, the task will be tested using a single set of data that is randomized on each run. In Evaluation mode, the task will be tested using 3 fixed sets of data, which remain the same across all runs and are identical for every team (for more information, check chapter 2.3.1 **Testing modes** in the **FPGA_Hackathon_2026_Project_Guide** document).

You earn 4 points for correctly processing each test vector. In the Development mode, this value simply indicates that the task passed the test. In the Evaluation mode, your total base score is calculated by multiplying the number of points by the number of test vectors (3), so you can earn up to 12 points.

Additional points (up to 5) may be awarded for resource utilization (for more information, check chapter 2.3.2 **Bonus Points** in the **FPGA_Hackathon_2026_Project_Guide** document).